

Internet programming, Section 1

Chicken Site Document

Yoo, J. H. 32212808

Department of Mobile Systems Engineering

Kim, K.Y. 32220393

Department of Mobile Systems Engineering

GitHub: <https://github.com/YooJunHyuk123/Chicken-site>

2025-12-01

Table of Contents

1. Introduction
2. Requirements
3. Design
4. Implementation
5. Verification
6. Maintenance
7. Conclusion

1. Introduction

A. Purpose of Development

This project aims to develop a user-participatory web application called Boneless Chicken Dex. Users can sign up and log in to register their favorite boneless chicken dishes, as well as browse and search chicken entries created by other users. The project is designed with an HTML, CSS, and JavaScript-based frontend and a backend architecture prepared for AWS integration.

2. Requirements

A. Functional Requirements

Auth
Provides user registration, login, and logout functionality.
Logged-in user
Logged-in users can register, edit, and delete chicken item, including: image URL, brand, style(fried, spicy, soy sauce, mala, cheese), name, spiciness level, sweetness level, crispiness level, and a short description.
Both logged-in and non-logged-in user
Both logged-in and non-logged-in users can search and view all registered chicken items.

B. Non-functional Requirements

Stability
The server should operate reliably.
Error handling
If an error occurs, an error message must be output.
Maintenance ability
The maintenance of the server should be easy.

3. Design

A. Architectural Design

Since this project uses Python Django, it has been modularized into a Model View Template (MTV) structure and does not change the architectural design separately.

B. User Interface (UI) & User Experience (UE) Design

Card type UI
Place all the main content in a round corner card to induce the gaze to be naturally centered.
Reactive web
The input form and result screen are designed to look optimized not only in desktop but also in mobile environments.

C. Database Design

The database uses Python Django's SQLite3 and is designed as follows.

User table
User ID
Password

Chicken table	
User (Foreign Key)	User
Image_url	Char Field
Brand	Char Field
Style	Char Field
Name	Char Field
Spiciness	Integer Field (1~5)
Sweetness	Integer Field (1~5)
Crispiness	Integer Field (1~5)
Description	Text Field

4. Implementation

A. Time

Planning stage
We established a core goal: "A web service for sharing and managing chicken information." Key features were listed, including membership registration/login, chicken card registration, and search/filter functions.
Design stage
Designed the UI configuration and data flow of the front-end pages (auth, chicken add, chicken list).
Implementation stage
Implemented the front-end and back-end and connected them respectively

B. Roles and Responsibility

	Yoo, J. H.	Kim, K. Y.
HTML	Write skeleton code Modify Django static HTML	Write skeleton code Connect with JS
CSS	Write skeleton code Make aesthetically with Gemini	Write skeleton code
JavaScript	Write skeleton code Connect with Django server	Write skeleton code Implement function with GPT
Backend	Write python Django code	-

C. Development Environment & Tools

Operating System	
Amazon Linux 2023	
Kernel	
Linux 6.1.156	
IDE (Integrated Development Environment) & Editor	
Vim 9.1	
Programming language	
Python 3.9.24, HTML, CSS, JavaScript	
Framework	
Django	Python Framework for Backend's Web Server Implementation
Build tool	
pip 21.3.1	
Data Base	
SQLite3	

D. Functional Description by Module

Frontend
index.html: Chicken list and search sign_up.html: User sign-up login.html: User login upload.html: Chicken registration style.css: All HTML pages share a common stylesheet script.js: All HTML pages share a common JavaScript file
Backend
admin.py: Configures the Django admin interface to manage models and data through a web-based UI. apps.py: Contains application configuration and metadata for a Django app. models.py: Defines the data structure (models) of the application, which maps to database tables. views.py: Handles the request-response cycle and returns appropriate responses, often rendered with templates. tests.py: Contains unit tests for testing the functionality of the application and ensuring code quality. asgi.py: Defines the ASGI (Asynchronous Server Gateway Interface) configuration for handling asynchronous web traffic. settings.py: Holds configuration settings for the Django project, such as database settings, installed apps, and middleware. urls.py: Defines the URL routing configuration, mapping URLs to views and handling HTTP requests. wsgi.py: Defines the WSGI (Web Server Gateway Interface) configuration for running the application in a production environment. manage.py: A command-line utility for managing the Django project, running commands like server start, migrations, and tests.

E. Key Algorithm Description & Code Snippet

```
{% load static %}
```

생략

```
<body>
  <header class="container">
    <h1>치킨 사이트</h1>
    <div class="header-actions" id="user-section">
      <!-- Buttons handled by JS -->
    </div>
  </header>

  <div class="container">
    <hr>
    <h2>치킨 검색</h2>
    <input type="text" id="searchInput" placeholder="치킨
이름 검색" />
    <button type="button" class="btn"
onclick="searchChicken()">검색</button>

    <hr>
    <h2>전체 치킨 목록</h2>
    <div id="chicken-list"></div>
  </div>

  <script src="{% static 'script.js' %}"></script>
</body>
```

생략

index.html serves as the main page that displays all chicken entries. The chicken list is loaded when the page loads, and users can search by chicken name. The search feature is implemented in JavaScript and works in a case-insensitive manner

```
{% load static %}
```

생략

```
<body>
    <header class="container">
        <h1>치킨 사이트</h1>
        <div class="header-actions" id="user-section"></div>
    </header>

    <div class="container">
        <h2>회원가입</h2>
        <input type="text" id="username" placeholder="아이디"
/>
        <input type="password" id="password1"
placeholder="비밀번호" />
        <input type="password" id="password2"
placeholder="비밀번호 확인" />
        <button type="button" class="btn"
onclick="signUpUser()">회원가입</button>
    </div>

    <script src="{% static 'script.js' %}"></script>
</body>
```

생략

sign_up.html and login.html handle user authentication. POST-based Fetch API requests are used to prevent personal data from appearing in URLs. Upon successful login, users are redirected to the main page (index.html).

```
{% load static %}
```

생략

```
<body>
    <header class="container">
        <h1>치킨 사이트</h1>
        <div class="header-actions" id="user-section"></div>
    </header>

    <div class="container">
        <h2>치킨 등록</h2>
        <input type="text" id="image_url" placeholder="이미지
URL" />
        <input type="text" id="brand" placeholder="브랜드" />
        <input type="text" id="style" placeholder="스타일" />
        <input type="text" id="name" placeholder="치킨 이름" />
        <input type="number" id="spiciness" placeholder="맵기"
/>
        <input type="number" id="sweetness" placeholder="단맛"
/>
        <input type="number" id="crispiness"
placeholder="바삭함" />
        <textarea id="description"
placeholder="설명"></textarea>
        <button type="button" class="btn"
onclick="createChicken()">등록</button>
    </div>

    <script src="{% static 'script.js' %}"></script>
</body>
```

생략

upload.html allows logged-in users to register chicken information. Users can enter the chicken's name, brand, style, flavor scores, description, and image data. When the register button is clicked, the data is sent to the backend via JavaScript.

```
// Login user
function loginUser() {
  const username =
document.getElementById('username').value;
  const password =
document.getElementById('password').value;

  fetch('/login/', {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-
urlencoded' },
    body:
`username=${encodeURIComponent(username)}&password=${encodeURIComponent(password)}`
  })
  .then(response => response.json())
  .then(data => {
    if (data.success) {
      alert(data.data.message);
      window.location.href = '/';
    } else {
      alert(data.error);
    }
  })
};
}
```

script.js is the core JavaScript file shared by all pages. It handles the following tasks:

- Authentication status checking and header UI control
- Login, logout, and sign-up processing
- Chicken registration requests
- Chicken list retrieval and search
- Dynamic UI updates based on server responses

This approach reduces code duplication and improves maintainability.

AI Usage
AI tools were used for the following purposes: • Page and feature structure design • HTML and JavaScript code generation • Implementation of login and sign-up logic • Debugging search functionality During the development of this project, the following questions were asked to the AI (ChatGPT, Gemini). • Web service structure suitable for backend integration • Criteria for separating HTML pages (index, login, signup, upload) • Implementation methods for login and sign-up functionality • Preventing personal information exposure in URLs • Implementation of chicken registration and listing features • Analyzing why the search function was not working correctly • Implementing case-insensitive search logic • Identifying limitations of image URL input and improving it to file upload handling In this project, the AI was utilized as a development assistant that supported feature design, debugging, code improvement, and documentation.

5. Verification

Test case	Input	Output	Stat
Stability	Each GET, POST request	Server's response	Pass
Error handling	Each POST request with wrong id or password	Print error message	Pass
UI/UX normal operation	Browser-based testing	Normal operation	Pass

6. Maintenance

Modular Design & Scalability
The functionality was separated into individual modules to improve scalability and maintainability. Each module follows the Single Responsibility Principle (SRP) by using MVT design pattern, minimizing the impact on other modules when modifications are made.
Error Handling
If an error occurs, print an error message.
Code Readability
Comments were added to all modules and functions to enhance code readability and understanding.

7. Conclusion

Through this project, we learned how to design and implement a full-stack web application using Django with a clear separation of frontend and backend roles. The main challenges were handling user authentication securely and implementing dynamic search functionality, which improved our understanding of HTTP requests and JavaScript–Django interaction. Although the project uses basic image URL input and SQLite, it can be improved by supporting file uploads, enhancing UI design, and deploying to a cloud environment. Overall, this project provided practical experience in modular web development and strengthened our problem-solving and collaboration skills.